QMWS Spring 2026: Introduction to Python for Data Analysis

Day One: Foundations

Gavin Klorfine & Deborah Laze

Section 1

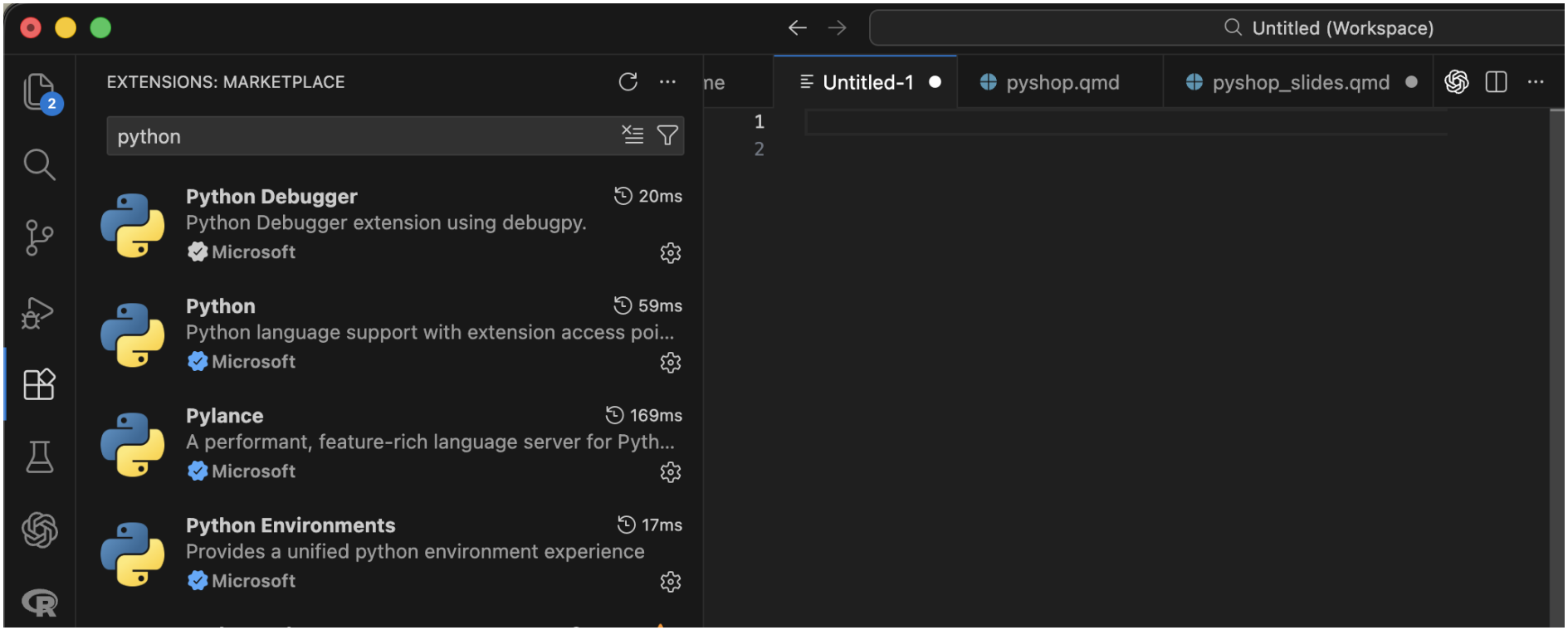
Installation

Visual Studio (VS) Code

- Installation link: https://code.visualstudio.com/download
- Integrated Development Environment (IDE)
- Can use it to code in many different languages
- Commonly used in both academia and industry

Visual Studio (VS) Code

- Once installed, go to the “Extensions” tab and search “Python”
- Install the below extensions:



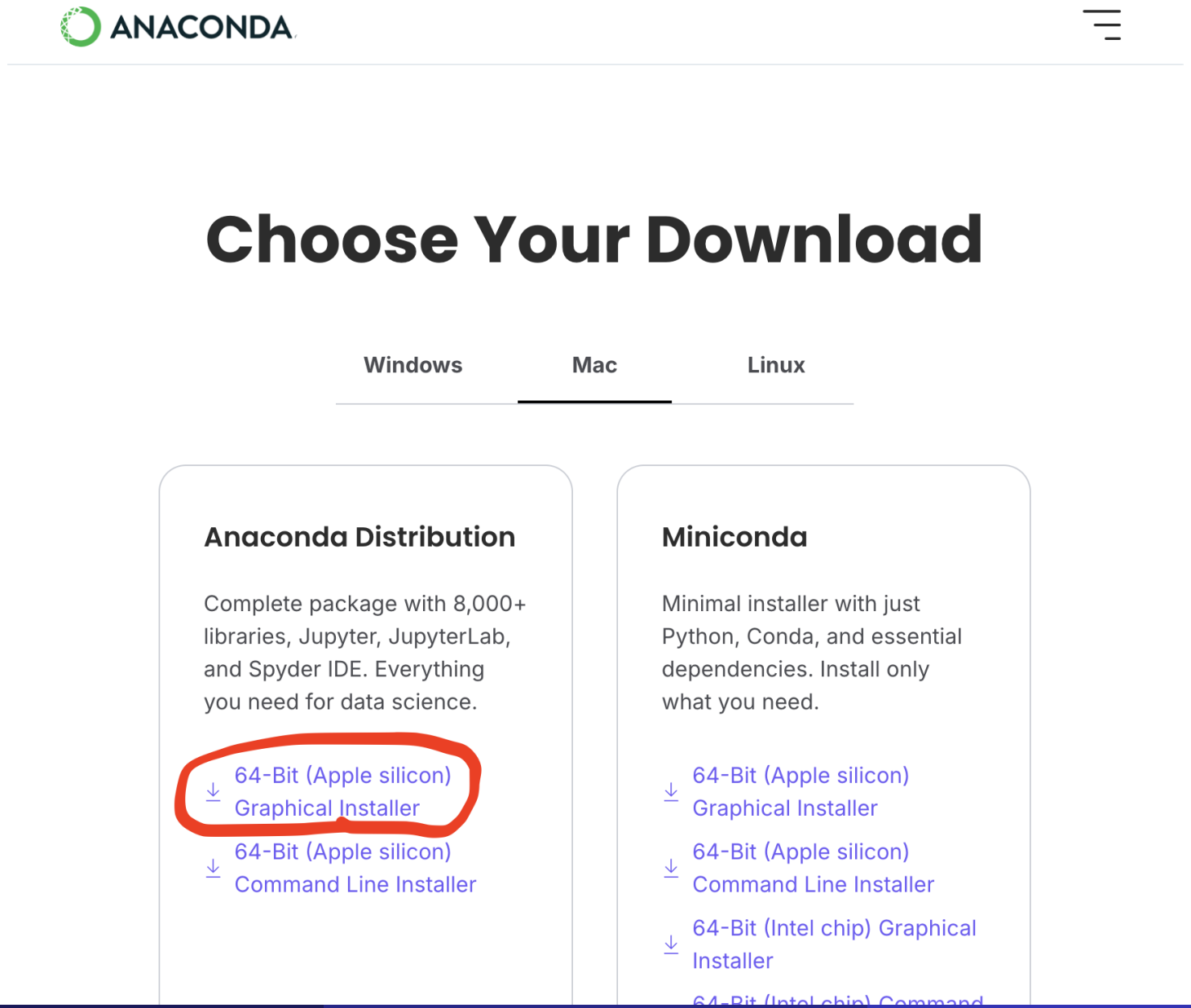
Anaconda

- A tool for configuring Python **environments**
  - More often than not, you need to install **packages** that allow you to code more easily/efficiently
  - Allows for version control
  - Issues can arise if environments are not used, for example:
    - If you install two packages that conflict with one another
    - If you install a package that requires a specific version of Python
  - Anaconda allows these issues to be isolated to a specific environment, rather than affecting your whole Python installation
    - Especially useful if you have more than one project going on

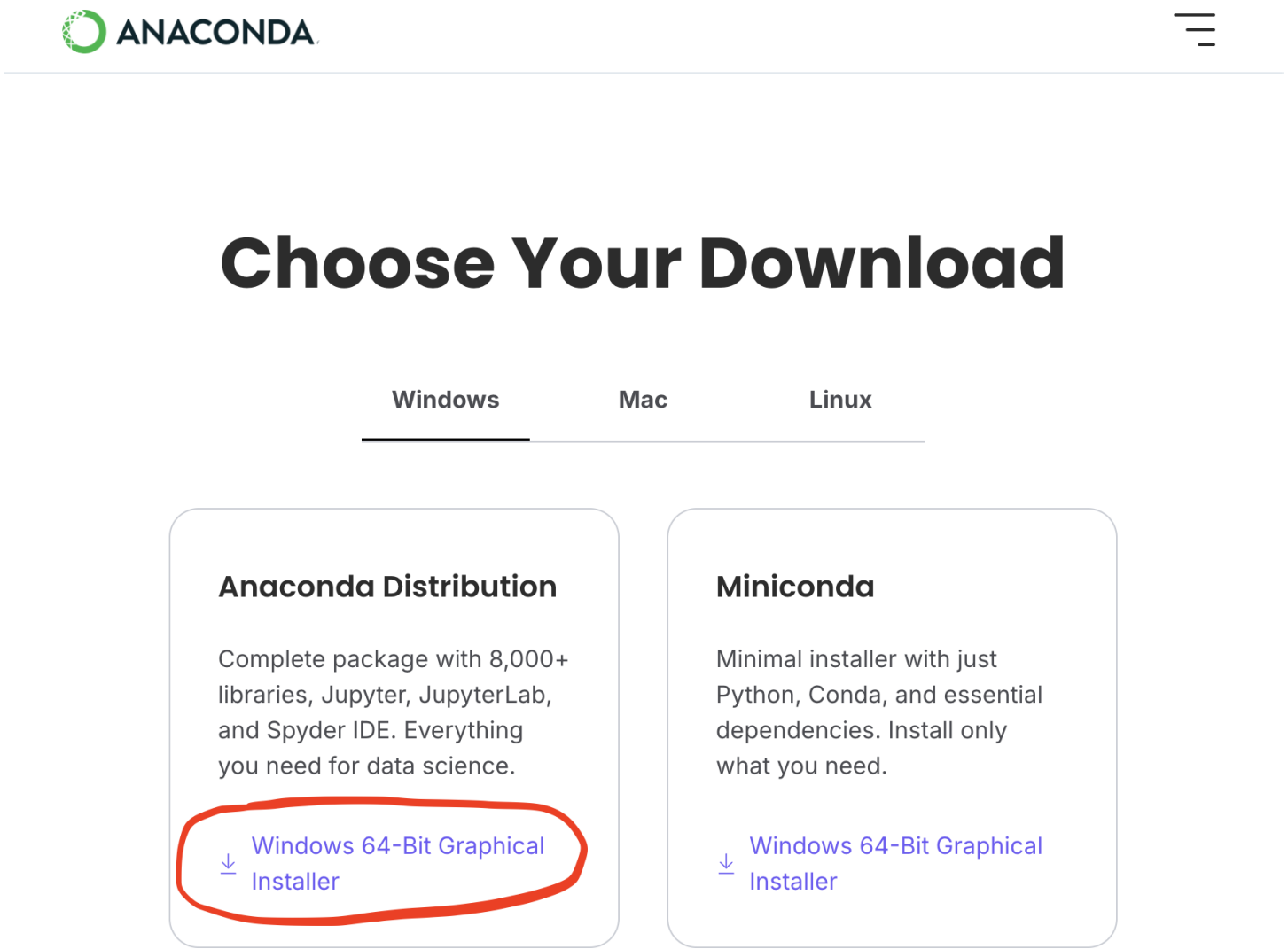
Anaconda

- Installation link: https://www.anaconda.com/download
- The simplest way to use Anaconda is via the **Navigator**
- To install:
  1. Make an account
  2. Install the full distribution via the "Graphical Installer"

- Installation link: https://www.anaconda.com/download
- On Mac:



- Installation link: https://www.anaconda.com/download
- On Windows:



Section 2

Setting Up a Conda Environment

Section 3

The Basics

Variables

- In programming, a variable is a space in computer memory where values may be stored

Defining and Printing Variables

```
x = 67

print(x)
```

```
67
```

Variables

Arithmetic

```
a = 1
b = 2

print(a + b)     # * for multiplication, / for division
```

```
3
```

Note

- Use # to make a **comment**

Variables

Updating Variables

```
x = 10
x = 20
x = 15

print (x)
```

```
15
```

Variable Types

1. `int` (integer; "whole number")
   - less memory
   - 1 is an `int`
2. `float` (real; "decimal number")
   - 1.2 is a `float`
   - 1.0 is a `float`
3. `str` (string; characters/words with no numerical meaning)
   - `"Hello world!"` is a string
   - `"416-676-6767"` is also a string
4. `bool` (boolean; can be `True` or `False`; more on this later)
5. `None`
   - A special type of variable that essentially means "nothing meaningful is stored"
   - Useful as a placeholder

Variable Types

Checking Type

```
x = 416
print(x, type(x))    # type() to check type
```

```
416 <class 'int'>
```

Converting Type

```
x = 416
x = str(x)       # str() to convert to string
print(x, type(x))
```

```
416 <class 'str'>
```

- Can also use `int()` and `float()` to convert to respective type

A Brief Note

- `print()`, `type()`, `str()`, etc. are all examples of **functions**
  - These are essentially pre-made chunks of code for you to use
  - More on this later!

What will the below code output?

```
x = "Variable"

print(type(x))
print(x)
```

Section 4

More Advanced Variable Types

Lists

- Multiple values can be stored in one variable. A list is one way to do this:

Defining Lists

```
x = [0, 1, 2, 3, 4]      # Define a list; square brackets
print(type(x))
```

```
<class 'list'>
```

Lists: Indexing

- To extract one or more values from a list, you **index** the list
- Python uses **zero-based indexing**
  - This means that the first element of a list is index 0, the second element is index 1, ...
  - Element x is index $x - 1$

Indexing Lists

```
x = [0, 1, 2, 3, 4]

print(x[0])      # Index first element of a list
print(x[-1])     # Index last element
```

```
0
4
```

Lists: Indexing

- You can also take more than one element, or a **slice**, of the list

Slicing Lists

```
x = [0, 1, 2, 3, 4]

print(x[0:2])   # first and second (but not third) elements
```

```
[0, 1]
```

Important

- `[inclusive:exclusive]`
  - The index on the *left-hand* side of the slice is *included*, whereas the index on the *right-hand* side of the slice is *excluded*

Lists: Indexing

Slicing Lists, continued

```
x = [0, 1, 2, 3, 4]

print(x[0:3])   # first three elements
print(x[0:1])   # a (silly) way to index just the first elemer
```

```
[0, 1, 2]
[0]
```

Lists: Length

- To find the length of a list:

len() function

```
x = [0, 1, 2, 3, 4]

print(len(x))   # The length of list x

# Another way to find the last element (not so silly!)
print(x[len(x) - 1])
```

```
5
4
```

Lists: Methods

- Lists have certain functions that can be applied to them, called **methods**
  - Methods are more general than this (e.g., string methods; more on these in a little bit!)

.append()

```
x = [0,1,2,3,4]

x.append(5)
print(x)
```

```
[0, 1, 2, 3, 4, 5]
```

- Adds an entry to a list

Lists: Methods

.extend()

```
x = [0,1,2,3,4]

x.extend([6,7,8,9])
print(x)
```

```
[0, 1, 2, 3, 4, 6, 7, 8, 9]
```

- Extend a list by "glueing" it to another list

Lists: Methods

Difference Between .append() and .extend()

```
x = [0,1,2,4]
y = [5,6,7,8]

x.append(y)
print(x)

x = [0,1,2,4]    # Redefine x
x.extend(y)
print(x)
```

```
[0, 1, 2, 4, [5, 6, 7, 8]]
[0, 1, 2, 4, 5, 6, 7, 8]
```

Lists: Changing Values

Changing a Value in a List

```
x = [0, 1, 2, 3, 4]

x[0] = 9
print(x)
```

```
[9, 1, 2, 3, 4]
```

Tuples (Briefly)

- Like lists, tuples contain multiple values
- Unlike lists, tuples are **immutable**; they cannot be changed once created.
  - Lists are *mutable*

Defining a Tuple

```
x = ("a", "tuple")  # Round brackets

print(x[0], x[1])
print(type(x))
```

```
a tuple
<class 'tuple'>
```

Tuples (Briefly)

Changing Values... ERROR!

```
x = ("a", "tuple")

x[0] = "Not a"
```

```
TypeError: 'tuple' object does not support item assignment
---------------------------------------------------------------
TypeError                                 Traceback (most rece
Cell In[173], line 3
      1 x = ("a", "tuple")
----> 3 x[0] = "Not a"
TypeError: 'tuple' object does not support item assignment
```

Tuples (Briefly)

- Tuples have special properties, one of which is below:

Unpacking Tuples

```
x, y = (6, 7)

print(x)
print(y)
```

```
6
7
```

Section 5

Strings

Strings: Basics

- Strings are a lot like lists in that they are **iterable**
  - For now, think of these (lists, strings) as a sequence of values

Strings are Like Lists!

```
cringe = "Hello world!"

print(cringe[0:5])
print(len(cringe))
```

```
Hello
12
```

Strings: Basics

- Strings are also kind of like tuples in that they are **immutable**

String Immutability

```
cringe = "Hello world!"
cringe[0:5] = "Goodbye"      # Error
```

```
TypeError: 'str' object does not support item assignment
---------------------------------------------------------------
TypeError                                 Traceback (most rece
Cell In[176], line 2
      1 cringe = "Hello world!"
----> 2 cringe[0:5] = "Goodbye"      # Error
TypeError: 'str' object does not support item assignment
```

Strings: Methods

- We can turn to string **methods** to "modify" strings when needed (they just create a new string "behind the scenes")

`.replace()`

```
cringe = "Hello world!"

print(cringe.replace("Hello", "Goodbye"))
```

```
Goodbye world!
```

Strings: Methods

.split()

```
meme_list = "baby gronk, 67, skibidi toilet, the rizzler"


print(meme_list.split(", ")) # Split on ", "; returns a list

print(meme_list.split(", ")[0]) # Index the list
```

```
['baby gronk', '67', 'skibidi toilet', 'the rizzler']
baby gronk
```

Strings: Methods

How might you index “the rizzler”?

Strings: Methods

How might you index "the rizzler"?

```
meme_list = "baby gronk, 67, skibidi toilet, the rizzler"

print(meme_list.split(", ")[3])
print(meme_list.split(", ")[-1])
print(meme_list.split(", ")[len(meme_list.split(", ")) - 1])
```

```
the rizzler
the rizzler
the rizzler
```

Strings: f-strings

- If you wanted to include variables within text, format strings as per the following:

f-string Example

```
x = 3.14159

print(f'Approximate value of pi is {x}')

print(f'To three decimal places is {x:.3f}')
```

```
Approximate value of pi is 3.14159
To three decimal places is 3.142
```

Strings: f-strings

- The string (surrounded by either single `''` or double `""` quotations) is preceded by `f`
- The variable is surrounded by curly braces `{}` within the string
- Formatting is applied to the variable *within the curly braces*
  - A `:` signals that the proceeding formatting code should be applied to the variable

f-string Formatting Basics

- `x:.3f`
  - Format variable `x`
  - Round to 3 decimal places
  - `f` means `float`

Section 6

Booleans

Booleans

- A boolean variable (type `bool`) can be either `True` (`1`) or `False` (`0`).

Booleans

```
x = True
y = 10
print(type(x), type(y))

print (y > 5)
print (y < 5)
print(type(y < 5))
```

```
<class 'bool'> <class 'int'>
True
False
<class 'bool'>
```

Booleans

Comparison Operators

- > greater than
- < less than
- >= greater than or equal to
- <= less than or equal to
- == equal to
- != not equal to

Logical Operators

- `and`
- `or`
- `not`